



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación

Asignatura: Desarrollo de Aplicaciones Móviles

NRC: 30738

Laboratorio 2

Desarrollo de una Aplicación Móvil con Principios de Diseño

Integrantes:

Cáceres Germán

Caiza Carlos

Campoverde Pablo

Mortensen Eduardo

Zurita Pablo

Docente: Ing. Doris Chicaiza MSc

Fecha: 4 de mayo de 2026

Índice

Índice de Algoritmos	ii
Índice de Anexos	ii
1 Introducción	1
1.1 Objetivos	1
1.1.1 Objetivo General	1
1.1.2 Objetivos Específicos	1
2 Marco Teórico	1
2.1 Flutter y Dart	1
2.2 Splash Screen	2
2.3 Patrón Arquitectónico MVC	2
2.4 Diseño Atómico (<i>Atomic Design</i>)	2
2.5 Navegación con Rutas Nombradas	2
2.6 Contraste: MVC versus Diseño Atómico	3
3 Metodología	3
3.1 Herramientas y Tecnologías Utilizadas	3
3.2 Estructura del Proyecto	3
4 Desarrollo	4
4.1 Splash Screen – Pantalla de Bienvenida	4
4.1.1 Descripción	4
4.1.2 Implementación con Diseño Atómico	4
4.2 Problema 1 – Venta y Factura	4
4.2.1 Descripción	4
4.2.2 Diagrama de Flujo	6
4.2.3 Pseudocódigo	7
4.2.4 Implementación MVC	7
4.3 Problema 4.9 – Cuenta de Ahorros	7
4.3.1 Descripción	7
4.3.2 Diagrama de Flujo	8
4.3.3 Pseudocódigo	9
4.3.4 Implementación MVC	9
4.4 Problema 4.10 – Promedios de Salones	9
4.4.1 Descripción	9
4.4.2 Diagrama de Flujo	10
4.4.3 Pseudocódigo	11
4.4.4 Implementación MVC	11
4.5 Problema 4.12 – Caja Registradora	11
4.5.1 Descripción	11
4.5.2 Diagrama de Flujo	12
4.5.3 Pseudocódigo	12
4.5.4 Implementación MVC	12
4.6 Problema 4.13 – Análisis de Ventas	13
4.6.1 Descripción	13

4.6.2	Diagrama de Flujo	13
4.6.3	Pseudocódigo	14
4.6.4	Implementación MVC	14
5	Conclusiones	14
6	Recomendaciones	16
	Referencias	17
	Anexos	18

Índice de figuras

1	Diagrama de flujo – Problema 1: Venta y Factura	6
2	Diagrama de flujo – Problema 4.9: Cuenta de Ahorros	8
3	Diagrama de flujo – Problema 4.10: Promedios de Salones	10
4	Diagrama de flujo – Problema 4.12: Caja Registradora	12
5	Diagrama de flujo – Problema 4.13: Análisis de Ventas	13

Índice de Algoritmos

1	Cálculo de Factura con IVA, Descuento y Comisión	7
2	Saldo de Cuenta de Ahorros con Capitalización Compuesta Mensual	9
3	Cálculo de Promedios de Salones y Escuela	11
4	Cálculo de Totales en Caja Registradora	12
5	Análisis y Clasificación de Ventas	14

Índice de Anexos

1	Splash screen – Pantalla de bienvenida	18
2	Menú principal con rutas nombradas	18
3	Problema 1 – Venta y Factura	19
4	Problema 4.9 – Cuenta de Ahorros	19
5	Problema 4.10 – Promedios de Salones	20
6	Problema 4.12 – Caja Registradora	20
7	Problema 4.13 – Análisis de Ventas	21

1. Introducción

El diseño de interfaces de usuario constituye uno de los pilares fundamentales en el desarrollo de aplicaciones móviles modernas. Una aplicación bien estructurada no solo debe resolver problemas de negocio, sino también ofrecer una experiencia visual coherente, intuitiva y estéticamente agradable para el usuario final.

El presente documento es el informe técnico del **Laboratorio 2 – Desarrollo de una Aplicación Móvil con Principios de Diseño**, desarrollado en la asignatura *Desarrollo de Aplicaciones Móviles* de la Universidad de las Fuerzas Armadas ESPE. La aplicación, denominada **Laboratorio 2**, fue construida con el *framework* **Flutter** y el lenguaje **Dart**, integrando el patrón arquitectónico **Modelo–Vista–Controlador (MVC)** con los principios del **Diseño Atómico** para estructurar los componentes de interfaz en jerarquías reutilizables.

La solución incluye: una pantalla de bienvenida animada (*splash screen*), un menú principal con navegación por rutas nombradas y cinco ejercicios prácticos de cálculo.

1.1 Objetivos

1.1.1 Objetivo General

Desarrollar una aplicación móvil multiplataforma con Flutter/Dart que aplique principios de diseño de interfaz (Diseño Atómico), el patrón MVC y navegación con rutas nombradas, para resolver cinco problemas de cálculo con lógica de negocio diferenciada.

1.1.2 Objetivos Específicos

- Implementar una pantalla de bienvenida (*splash screen*) con animaciones de entrada (fundido y deslizamiento) que transite automáticamente al menú principal.
- Aplicar el patrón MVC para separar las responsabilidades de datos, lógica de negocio y presentación en cada ejercicio.
- Organizar la capa de presentación siguiendo el Diseño Atómico: átomos, moléculas, organismos y páginas.
- Implementar la navegación entre pantallas mediante el sistema de rutas nombradas de Flutter.
- Resolver cinco problemas de cálculo que cubran estructuras secuenciales, condicionales e iterativas.

2. Marco Teórico

2.1 Flutter y Dart

Flutter es un *framework* de código abierto creado por Google para el desarrollo de aplicaciones nativas multiplataforma (iOS, Android, Web, Desktop) a partir de una única base de código. Utiliza el lenguaje Dart y un motor de renderizado propio (*Skia/Impeller*) que garantiza una apariencia visual consistente entre plataformas [1].

Dart es un lenguaje orientado a objetos, fuertemente tipado y compilado, con soporte para programación asíncrona mediante `async/await` y compilación *Ahead-of-Time* (AOT) [2]. Sus características más relevantes son:

- **Tipado estático con *null safety***: garantiza que las variables no puedan ser `null` salvo declaración explícita.

- **Compilación AOT y JIT:** AOT para producción y JIT para desarrollo (*Hot Reload*).

2.2 Splash Screen

Un *splash screen* (pantalla de bienvenida) es la primera pantalla visible al iniciar una aplicación móvil. Presenta el logotipo y la identidad visual mientras se carga el contenido inicial. En Flutter se implementa como la ruta inicial (`initialRoute: '/'`), utilizando `AnimationController` para efectos de entrada y `Future.delayed` para la transición automática al menú principal mediante `Navigator.pushReplacementNamed`.

2.3 Patrón Arquitectónico MVC

El patrón **Modelo–Vista–Controlador** (MVC) separa la aplicación en tres capas con responsabilidades bien definidas [3]:

- **Modelo:** encapsula los datos y la lógica de negocio. Es independiente de la interfaz gráfica.
- **Vista:** responsable únicamente de la presentación. No procesa ni almacena datos.
- **Controlador:** intermediario entre Modelo y Vista; recibe entradas del usuario, delega al Modelo y actualiza la Vista.

En Flutter, MVC se implementa organizando las carpetas `model/`, `view/` y `controller/` dentro de `lib/`.

2.4 Diseño Atómico (*Atomic Design*)

El **Diseño Atómico**, propuesto por Brad Frost [4], organiza los componentes de interfaz en cinco niveles:

1. **Átomos:** componentes mínimos e indivisibles (`TextField`, `Icon`, `Text`).
2. **Moléculas:** combinación de átomos con una función concreta (campo de entrada con etiqueta).
3. **Organismos:** grupos de moléculas que forman una sección completa (formulario + resultado).
4. **Plantillas:** esqueleto de la pantalla sin contenido real.
5. **Páginas:** instancias concretas con contenido real y navegación activa.

En la aplicación, cada archivo de vista implementa los cuatro primeros niveles como clases privadas (prefijo `_`) dentro del mismo archivo.

2.5 Navegación con Rutas Nombradas

Flutter gestiona la navegación mediante una pila (*stack*) de rutas. Con rutas nombradas, se define un mapa `routes` en `MaterialApp` y se navega con:

- `Navigator.pushNamed`: agrega una nueva pantalla a la pila (permite regresar con *back*).
- `Navigator.pushReplacementNamed`: reemplaza la pantalla actual (usado en el splash para que no sea alcanzable con el botón atrás).
- `Navigator.pop`: elimina la pantalla actual y regresa a la anterior.

2.6 Contraste: MVC versus Diseño Atómico

Cuadro 1: Comparativa entre MVC y Diseño Atómico en el contexto de Flutter

Criterio	MVC	Diseño Atómico
Enfoque	Separación de capas lógicas	Composición jerárquica de UI
Nivel de abstracción	Arquitectura de la aplicación	Organización interna de la vista
Unidad base	Módulo funcional (model/controller/view)	Widget atómico
Reutilización	Por capa y por función	Por nivel de composición
Estructura en Flutter	Carpetas <code>model/</code> , <code>view/</code> , <code>controller/</code>	Átomos, moléculas y organismos dentro de cada vista

Ambos paradigmas son complementarios: MVC rige la organización de archivos a nivel de carpetas, mientras que Diseño Atómico guía la composición interna de cada Vista.

3. Metodología

El desarrollo de la aplicación siguió un proceso iterativo e incremental:

1. **Análisis de requisitos:** identificación de los cinco ejercicios, sus entradas, salidas y reglas de negocio.
2. **Diseño del splash screen:** definición de animaciones, identidad visual y transición automática.
3. **Diseño arquitectónico:** estructura MVC con carpetas `model/`, `view/`, `controller/`.
4. **Modelado:** clases Dart puras con lógica de cálculo, sin dependencias de Flutter.
5. **Implementación de controladores:** parseo y validación de entradas del usuario.
6. **Diseño de vistas:** construcción de interfaces aplicando Diseño Atómico (átomos → moléculas → organismos → páginas).
7. **Integración:** configuración de rutas nombradas en `main.dart` y menú principal.
8. **Pruebas:** verificación manual en emulador Android.

3.1 Herramientas y Tecnologías Utilizadas

- **Flutter SDK 3.x / Dart 3.x:** *framework* y lenguaje con *null safety*.
- **Visual Studio Code** con extensiones Flutter y Dart.
- **Android Emulator:** entorno de emulación y pruebas.
- **Git / GitHub:** control de versiones.

3.2 Estructura del Proyecto

```
lib/
  main.dart                <- Punto de entrada y rutas nombradas
```

```

model/
  venta_model.dart          <- Factura con IVA, descuento y comisión
  banco_model.dart          <- Cuenta de ahorros compuesta mensual
  salon_model.dart          <- Promedios por salón y escuela
  caja_model.dart           <- Caja registradora por denominaciones
  ventas_analisis_model.dart <- Análisis de ventas por categorías
controller/
  venta_controller.dart
  banco_controller.dart
  salon_controller.dart
  caja_controller.dart
  ventas_analisis_controller.dart
view/
  splash_view.dart          <- Pantalla de bienvenida animada
  menu_view.dart           <- Menú principal con rutas nombradas
  venta_view.dart
  banco_view.dart
  salon_view.dart
  caja_view.dart
  ventas_analisis_view.dart

```

4. Desarrollo

4.1 Splash Screen – Pantalla de Bienvenida

4.1.1 Descripción

La pantalla de bienvenida es la primera pantalla visible al iniciar la aplicación. Muestra el logotipo y el nombre de la aplicación sobre un fondo degradado azul, con animaciones de fundido (*fade-in*) y deslizamiento (*slide-up*) durante 1500 ms. Tras 3 segundos, transita automáticamente al menú principal.

4.1.2 Implementación con Diseño Atómico

- **Átomo** _SplashLogo: icono calculate_rounded sobre un círculo semitransparente.
- **Átomo** _SplashTitle: título “Laboratorio 2”, subtítulo y nombre del grupo.
- **Molécula** _SplashLoader: CircularProgressIndicator con texto “Cargando...”.
- **Organismo** _SplashCard: composición vertical de logo, título y loader.
- **Página** SplashView: StatefulWidget con AnimationController, FadeTransition + SlideTransition y Future.delayed(3s) → push ReplacementNamed('/menu').

4.2 Problema 1 – Venta y Factura

4.2.1 Descripción

El vendedor ingresa una lista de productos (nombre, precio, cantidad) y su sueldo base. La aplicación calcula la factura completa: subtotal, IVA, descuento por volumen, total final y el sueldo total del vendedor incluyendo comisión.

Reglas de negocio:

$$\text{subtotal} = \sum_{i=1}^n \text{precio}_i \times \text{cantidad}_i \quad (1)$$

$$\text{IVA} = \text{subtotal} \times 0,15 \quad (2)$$

$$\text{subtotalIVA} = \text{subtotal} + \text{IVA} \quad (3)$$

$$\text{descuento} = \begin{cases} \text{subtotalIVA} \times 0,20 & \text{si } \text{subtotalIVA} > 2000 \\ 0 & \text{caso contrario} \end{cases} \quad (4)$$

$$\text{totalFinal} = \text{subtotalIVA} - \text{descuento} \quad (5)$$

$$\text{comisión} = \text{subtotal} \times 0,05 \quad (6)$$

$$\text{sueldoTotal} = \text{sueldoBase} + \text{comisión} \quad (7)$$

4.2.2 Diagrama de Flujo

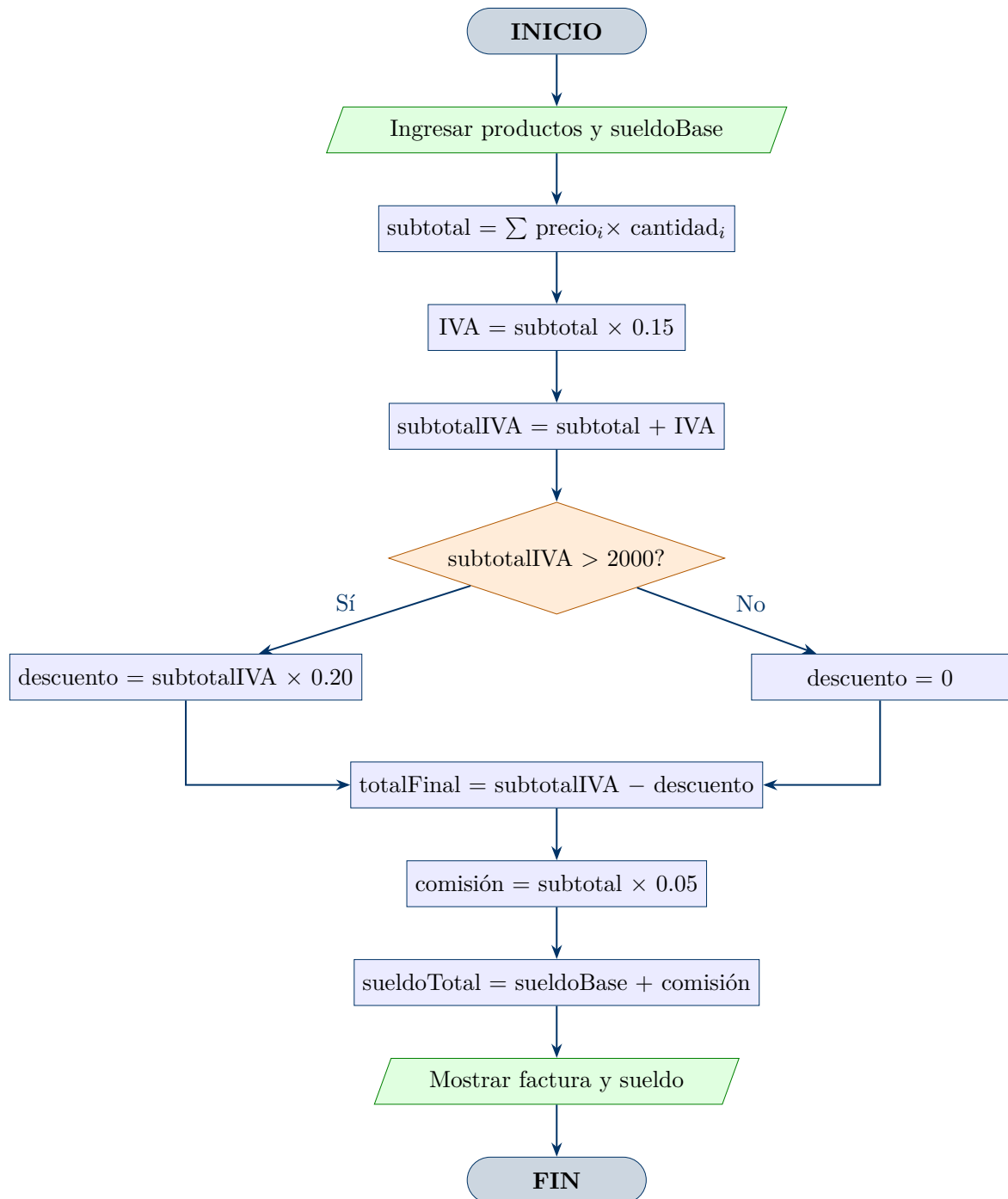


Figura 1: Diagrama de flujo – Problema 1: Venta y Factura

4.2.3 Pseudocódigo

Algoritmo 1 Cálculo de Factura con IVA, Descuento y Comisión

Require: productos = $[(p_i, c_i)]$, sueldoBase ≥ 0

Ensure: totalFinal, sueldoTotal

```

1: subtotal  $\leftarrow \sum p_i \times c_i$ 
2: IVA  $\leftarrow subtotal \times 0,15$ 
3: subtotalIVA  $\leftarrow subtotal + IVA$ 
4: if subtotalIVA > 2000 then
5:   descuento  $\leftarrow subtotalIVA \times 0,20$ 
6: else
7:   descuento  $\leftarrow 0$ 
8: end if
9: totalFinal  $\leftarrow subtotalIVA - descuento$ 
10: comision  $\leftarrow subtotal \times 0,05$ 
11: sueldoTotal  $\leftarrow sueldoBase + comision$ 
12: MOSTRAR subtotal, IVA, descuento, totalFinal, sueldoTotal

```

4.2.4 Implementación MVC

- **Modelo** (venta_model.dart): clases ProductoItem (nombre, precio, cantidad, getter subtotal) y FacturaResultado con método estático calcular(List<ProductoItem>, sueldoBase).
- **Controlador** (venta_controller.dart): valida y parsea las entradas de texto del usuario, construye la lista de productos y delega al modelo.
- **Vista** (venta_view.dart): StatefulWidget con AppBar azul. Lista dinámica de productos. Átomos: campos de texto. Moléculas: fila de producto. Organismo: formulario + resultados. Página: VentaPage.

4.3 Problema 4.9 – Cuenta de Ahorros

4.3.1 Descripción

Se deposita una cantidad fija mensual en una cuenta de ahorros con capitalización compuesta mensual a una tasa nominal anual del 10 %. La aplicación calcula el saldo acumulado al final de cada año durante N años.

Fórmula de capitalización compuesta mensual:

$$r_m = \frac{0,10}{12}, \quad S_m = (S_{m-1} + D) \times (1 + r_m)$$

donde D es el depósito mensual, S_m el saldo al mes m y r_m la tasa mensual.

4.3.2 Diagrama de Flujo

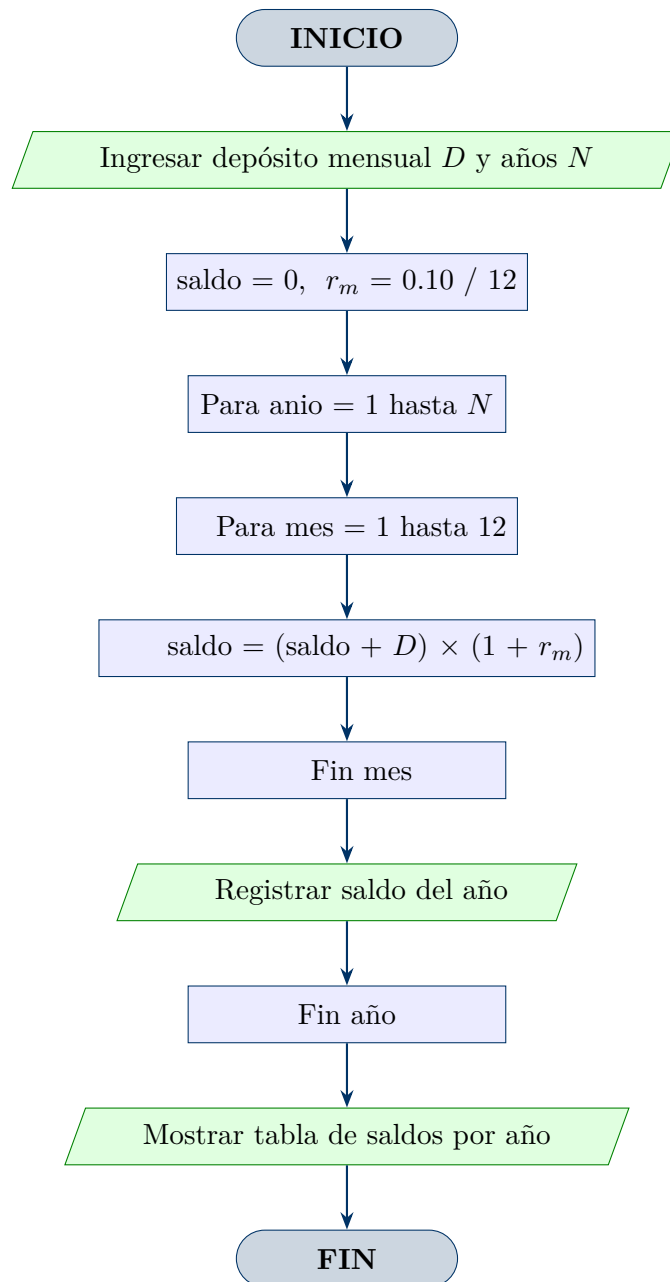


Figura 2: Diagrama de flujo – Problema 4.9: Cuenta de Ahorros

4.3.3 Pseudocódigo

Algoritmo 2 Saldo de Cuenta de Ahorros con Capitalización Compuesta Mensual

Require: $D > 0$, $N \geq 1$

Ensure: tabla de saldos por año

```

1:  $saldo \leftarrow 0$ 
2:  $r_m \leftarrow 0,10 \div 12$ 
3: for  $anio \leftarrow 1$  hasta  $N$  do
4:   for  $mes \leftarrow 1$  hasta 12 do
5:      $saldo \leftarrow (saldo + D) \times (1 + r_m)$ 
6:   end for
7:   REGISTRAR ( $anio$ ,  $saldo$ )
8: end for
9: MOSTRAR tabla de resultados
  
```

4.3.4 Implementación MVC

- **Modelo** (banco_model.dart): clase BancoAnioResultado (año, saldoFinal) y BancoModel.calcular(depositoMensual, nAnios) que retorna List<BancoAnioResultado>.
- **Controlador** (banco_controller.dart): parsea y valida las entradas de texto y delega el cálculo al modelo.
- **Vista** (banco_view.dart): AppBar índigo. Tabla de resultados con año y saldo formateado. BancoPage.

4.4 Problema 4.10 – Promedios de Salones

4.4.1 Descripción

Se tienen S salones, cada uno con n_i alumnos. Para cada salón se calcula el promedio de edades y finalmente se obtiene el promedio general de toda la escuela.

Fórmulas:

$$\bar{x}_i = \frac{\sum_{j=1}^{n_i} e_{ij}}{n_i}, \quad \bar{x}_{\text{escuela}} = \frac{\sum_{i=1}^S \sum_{j=1}^{n_i} e_{ij}}{\sum_{i=1}^S n_i}$$

4.4.2 Diagrama de Flujo

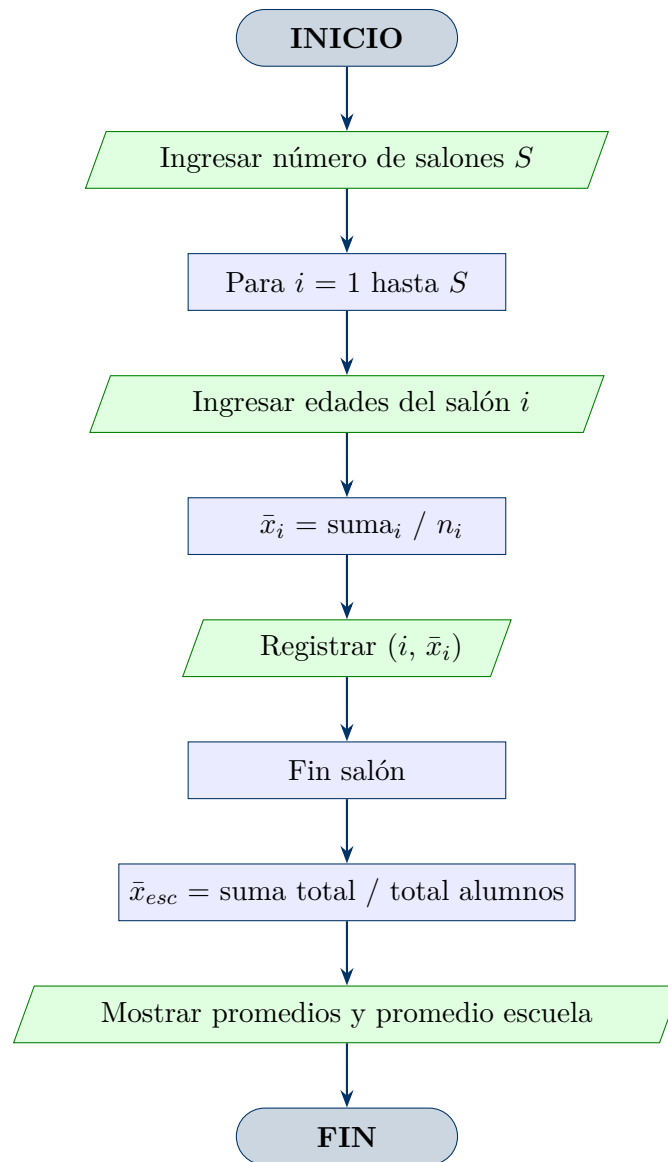


Figura 3: Diagrama de flujo – Problema 4.10: Promedios de Salones

4.4.3 Pseudocódigo

Algoritmo 3 Cálculo de Promedios de Salones y Escuela

Require: $S \geq 1$, edades $e_{ij} \geq 0$

Ensure: $\bar{x}_i$ para cada salón, $\bar{x}_{escuela}$

```

1:  $totalEdades \leftarrow 0$ ;  $totalAlumnos \leftarrow 0$ 
2: for  $i \leftarrow 1$  hasta  $S$  do
3:   LEER  $[e_{i1}, e_{i2}, \dots, e_{in_i}]$ 
4:    $\bar{x}_i \leftarrow \sum e_{ij} \div n_i$ 
5:   REGISTRAR  $(i, \bar{x}_i)$ 
6:    $totalEdades \leftarrow totalEdades + \sum e_{ij}$ 
7:    $totalAlumnos \leftarrow totalAlumnos + n_i$ 
8: end for
9:  $\bar{x}_{escuela} \leftarrow totalEdades \div totalAlumnos$ 
10: MOSTRAR promedios por salón y  $\bar{x}_{escuela}$ 

```

4.4.4 Implementación MVC

- **Modelo** (`salon_model.dart`): clase `SalonResultado` (número, edades, promedio); `SalonModel.calcular(List<List<double>)` y `SalonModel.promedioEscuela(List<SalonResultado>)`.
- **Controlador** (`salon_controller.dart`): parseo de las edades ingresadas como texto separado por comas.
- **Vista** (`salon_view.dart`): formulario en dos pasos (número de salones $\rightarrow$ edades). AppBar teal. `SalonPage`.

4.5 Problema 4.12 – Caja Registradora

4.5.1 Descripción

El cajero registra la cantidad de billetes y monedas de cada denominación en caja. La aplicación calcula el subtotal por denominación y el total general.

Fórmula:

$$\text{subtotal}_i = \text{denominación}_i \times \text{cantidad}_i, \quad \text{total} = \sum_{i=1}^n \text{subtotal}_i$$

4.5.2 Diagrama de Flujo

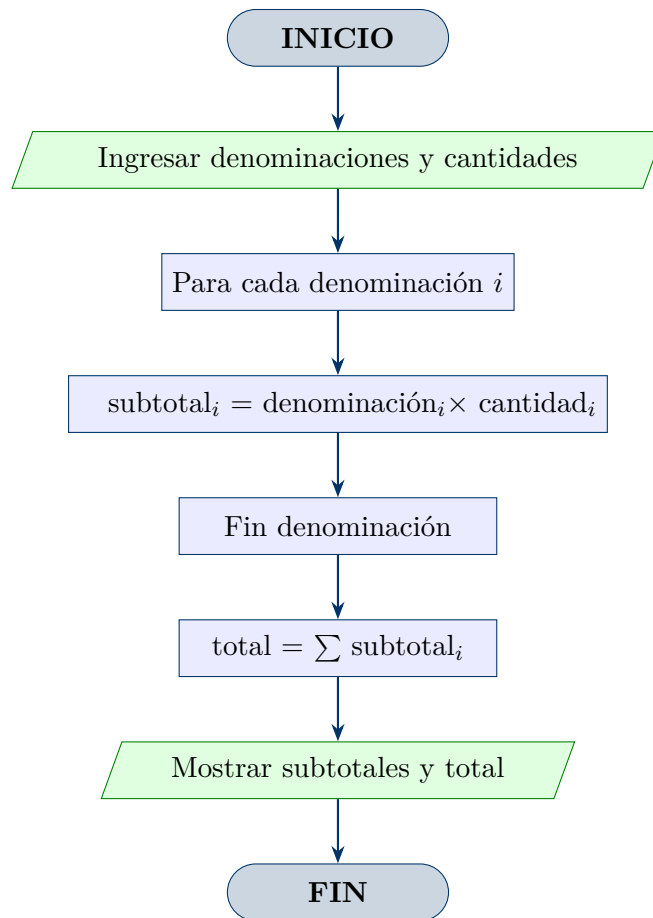


Figura 4: Diagrama de flujo – Problema 4.12: Caja Registradora

4.5.3 Pseudocódigo

Algoritmo 4 Cálculo de Totales en Caja Registradora

Require: denominaciones $d_i > 0$, cantidades $c_i \geq 0$

Ensure: subtotales por denominación y total general

```

1: total ← 0
2: for cada denominación  $i$  do
3:   subtotali ←  $d_i \times c_i$ 
4:   total ← total + subtotali
5: end for
6: MOSTRAR subtotali para cada  $i$  y total
  
```

4.5.4 Implementación MVC

- **Modelo** (caja_model.dart): clase BilleteMoneda (denominación, cantidad, getter subtotal) y CajaModel.calcularTotal(List<BilleteMoneda>).
- **Controlador** (caja_controller.dart): construye la lista de denominaciones a partir de las cantidades ingresadas por el usuario.

- **Vista** (`caja_view.dart`): tabla de denominaciones fijas con campos editables. AppBar naranja. CajaPage.

4.6 Problema 4.13 – Análisis de Ventas

4.6.1 Descripción

Se ingresan N montos de ventas. Cada venta se clasifica en una de tres categorías según su monto. Se reporta el conteo, el subtotal y el porcentaje de cada categoría, más el total global.

Criterio de clasificación:

$$\text{categoría} = \begin{cases} \text{Baja} & \text{si monto} \leq 10\,000 \\ \text{Media} & \text{si } 10\,000 < \text{monto} < 20\,000 \\ \text{Alta} & \text{si monto} \geq 20\,000 \end{cases}$$

4.6.2 Diagrama de Flujo

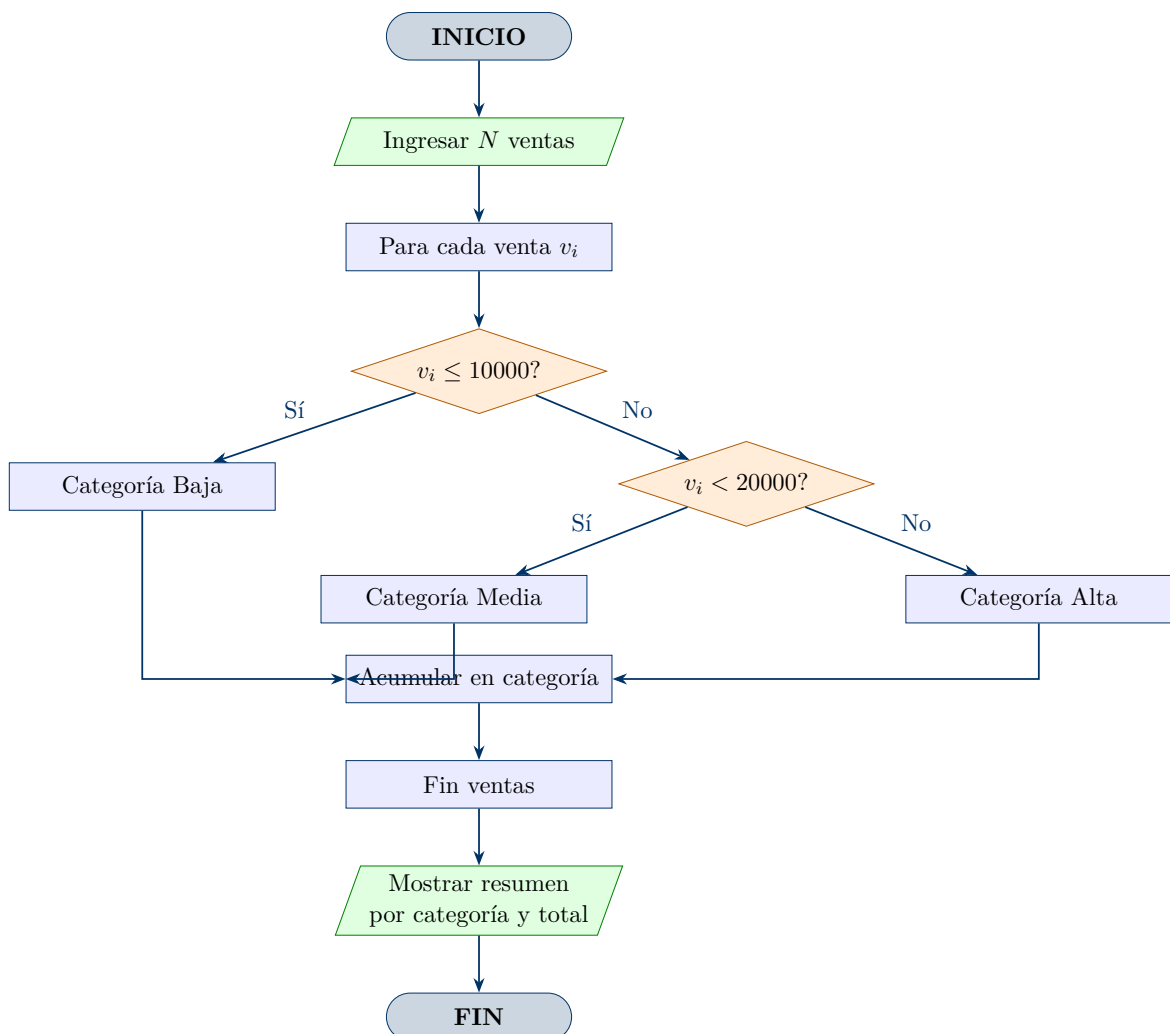


Figura 5: Diagrama de flujo – Problema 4.13: Análisis de Ventas

4.6.3 Pseudocódigo

Algoritmo 5 Análisis y Clasificación de Ventas

Require: $N \geq 1$, montos $v_i \geq 0$

Ensure: conteo y total por categoría, total global

```

1:  $counts[0..2] \leftarrow 0$ ;  $totals[0..2] \leftarrow 0$ 
2: for  $i \leftarrow 1$  hasta  $N$  do
3:   LEER  $v_i$ 
4:   if  $v_i \leq 10000$  then
5:      $cat \leftarrow 0$ 
6:   else if  $v_i < 20000$  then
7:      $cat \leftarrow 1$ 
8:   else
9:      $cat \leftarrow 2$ 
10:  end if
11:   $counts[cat] \leftarrow counts[cat] + 1$ 
12:   $totals[cat] \leftarrow totals[cat] + v_i$ 
13: end for
14:  $totalGlobal \leftarrow totals[0] + totals[1] + totals[2]$ 
15: MOSTRAR resumen por categoría y  $totalGlobal$ 

```

4.6.4 Implementación MVC

- **Modelo** (ventas_analisis_model.dart): clase `VentaItem` con getter `categoria`; `CategoriaResumen` (nombre, cantidad, total); `VentasAnalisisResultado`.`calcular(List<VentaItem>)`.
- **Controlador** (ventas_analisis_controller.dart): método `parsearVenta(String)` que valida y convierte el texto ingresado a `VentaItem`.
- **Vista** (ventas_analisis_view.dart): lista dinámica de entradas, resultados color-codificados por categoría (verde/naranja/azul). AppBar morado. `VentasAnalisisPage`.

5. Conclusiones

1. La implementación del **splash screen** con `AnimationController`, `FadeTransition` y `SlideTransition` demostró que Flutter provee herramientas nativas suficientes para construir experiencias de bienvenida profesionales sin dependencias externas, mejorando significativamente la primera impresión del usuario.
2. La combinación del patrón **MVC** con el **Diseño Atómico** resultó coherente y efectiva: MVC rige la separación de responsabilidades a nivel de archivos y carpetas, mientras que el Diseño Atómico estructura internamente cada Vista en componentes de interfaz progresivamente más complejos (átomo $\rightarrow$ molécula $\rightarrow$ organismo $\rightarrow$ página).
3. El sistema de **rutas nombradas** de Flutter (`routes`, `pushNamed`, `pushReplacementNamed`) simplificó la gestión de la navegación, garantizando que el splash screen no sea alcanzable desde el menú mediante el botón atrás al usar `pushReplacementNamed`.

4. Los cinco ejercicios cubrieron los principales paradigmas de programación estructurada: secuencial (Problema 1, 4.12), condicional (Problema 1, 4.13) e iterativo (Problema 4.9, 4.10, 4.13), validando la aplicabilidad del patrón MVC en problemas de diferente complejidad lógica.
5. El uso de clases Dart puras en la capa **Modelo** — sin ninguna dependencia de Flutter — garantiza que la lógica de negocio sea completamente portátil y testeable de forma unitaria, separando claramente el dominio del problema de los detalles de presentación.

6. Recomendaciones

1. Se recomienda migrar la gestión de estado del patrón **MVC** manual actual hacia una solución reactiva como **Provider** o **Riverpod**, lo que facilitará la sincronización automática entre la capa Modelo y la Vista a medida que la aplicación crezca en complejidad.
2. Para garantizar la calidad del código a largo plazo, se sugiere incorporar **pruebas unitarias** en la capa Modelo y **pruebas de widgets** en la capa Vista desde etapas tempranas del desarrollo, aprovechando que los modelos Dart puros son directamente testeables sin necesidad del framework Flutter.
3. Implementar **validación de entradas** en los formularios de las vistas (p. ej., campos numéricos, rangos permitidos) mediante el widget **Form** y **TextFormField** con **validator**, con el fin de prevenir errores en tiempo de ejecución causados por datos incorrectos proporcionados por el usuario.
4. Se aconseja aplicar un **tema centralizado** (**ThemeData**) en el **MaterialApp** para definir colores, tipografías y estilos una sola vez, eliminando la duplicación de estilos dispersos en las vistas individuales y facilitando futuros cambios de branding.
5. Para mejorar la **experiencia de usuario**, se recomienda añadir retroalimentación visual en las operaciones de cálculo (p. ej., indicadores de carga, animaciones de transición entre vistas) y mensajes de error descriptivos mediante **SnackBar** o **AlertDialog**, en lugar de mostrar resultados vacíos o sin contexto.
6. Conforme la aplicación incorpore más módulos, se sugiere evaluar el uso de **rutas con argumentos tipados** o paquetes de navegación declarativa como **GoRouter**, los cuales ofrecen mejor soporte para deep linking, manejo del historial y navegación anidada en aplicaciones de mediana y gran escala.

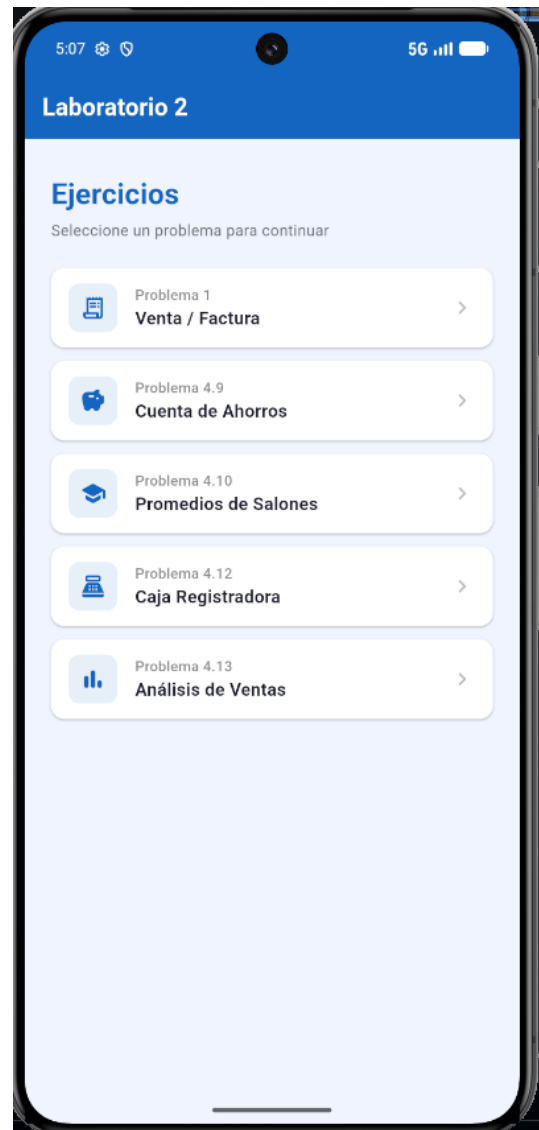
Referencias

- [1] Google LLC. (2024). *Flutter – Build apps for any screen*. Flutter Documentation. Recuperado de <https://flutter.dev/docs>
- [2] Google LLC. (2024). *Dart programming language*. Dart Documentation. Recuperado de <https://dart.dev/guides>
- [3] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. ISBN: 978-0-201-63361-0.
- [4] Frost, B. (2016). *Atomic Design*. Brad Frost Web. Recuperado de <https://atomicdesign.bradfrost.com>
- [5] Google LLC. (2024). *Navigation and routing – Named routes*. Flutter Documentation. Recuperado de <https://docs.flutter.dev/cookbook/navigation/named-routes>
- [6] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. ISBN: 978-0-13-468599-1.
- [7] Windmill, E. (2020). *Flutter in Action*. Manning Publications. ISBN: 978-1-61729-571-8.

Anexos



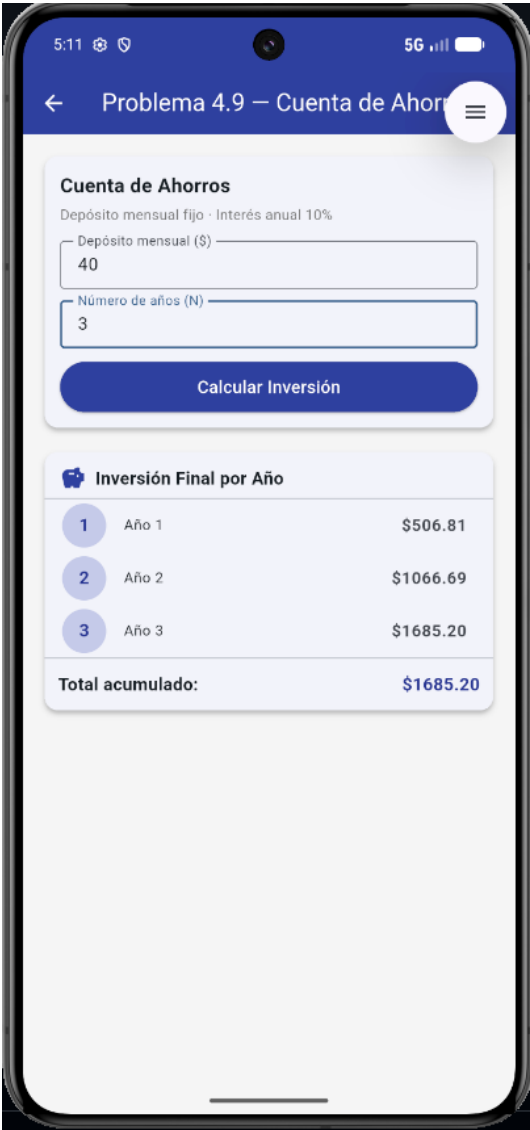
Anexo 1: Splash screen – Pantalla de bienvenida



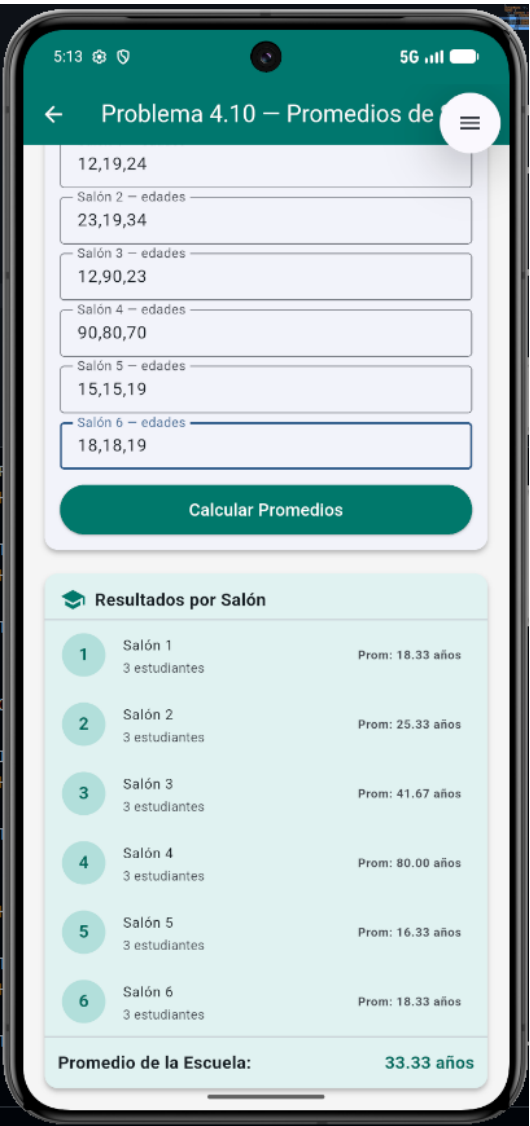
Anexo 2: Menú principal con rutas nombradas



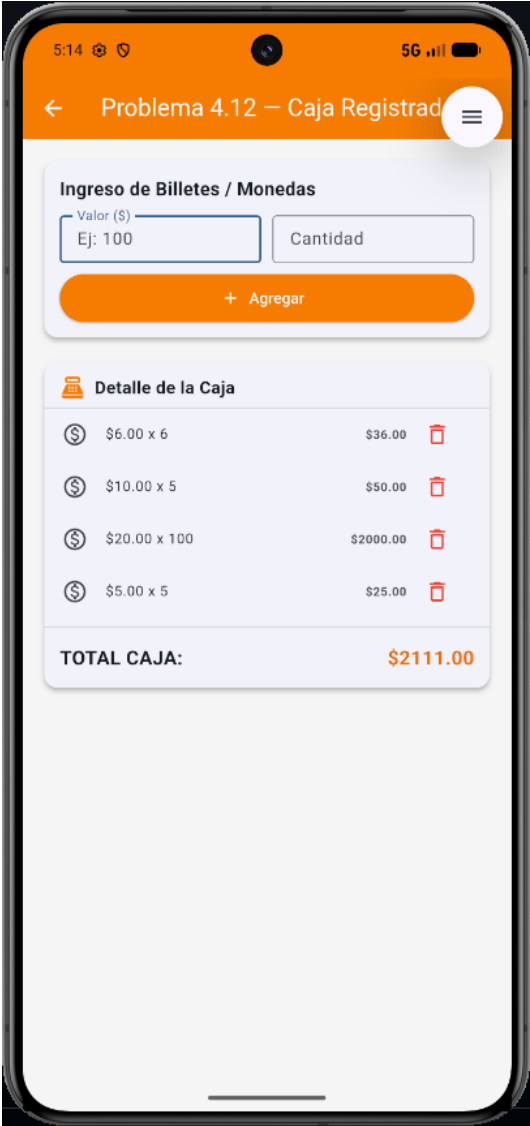
Anexo 3: Problema 1 – Venta y Factura



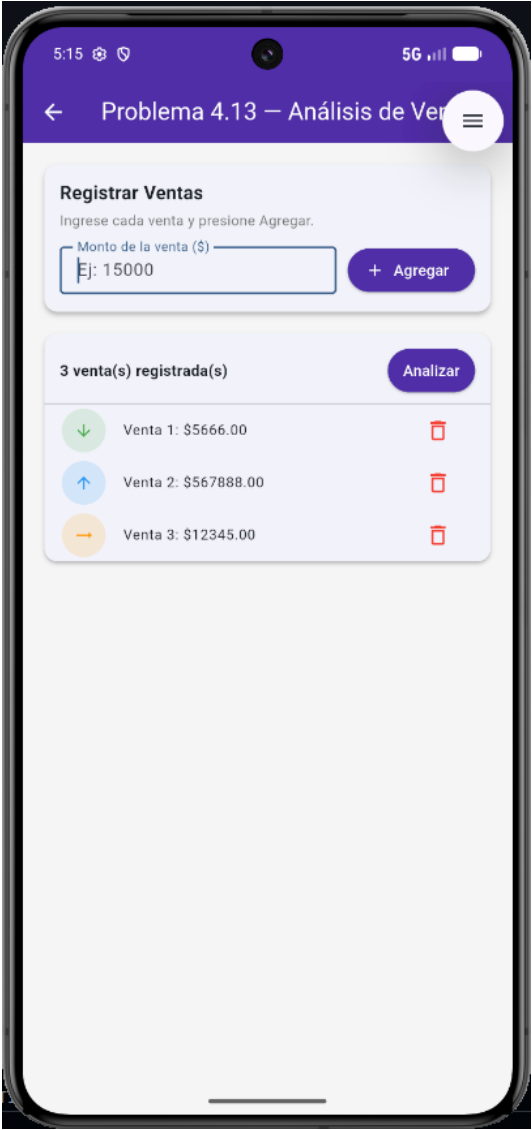
Anexo 4: Problema 4.9 – Cuenta de Ahorros



Anexo 5: Problema 4.10 – Promedios de Salones



Anexo 6: Problema 4.12 – Caja Registradora



Anexo 7: Problema 4.13 – Análisis de Ventas